Taylor & Francis
Taylor & Francis Group



International Journal of Production Research

ISSN: 0020-7543 (Print) 1366-588X (Online) Journal homepage: www.tandfonline.com/journals/tprs20

Production control with Reinforcement Learning for a matrix-structured production system

L. M. Steinbacher, T. Wegmann & M. Freitag

To cite this article: L. M. Steinbacher, T. Wegmann & M. Freitag (2025) Production control with Reinforcement Learning for a matrix-structured production system, International Journal of Production Research, 63:11, 4114-4136, DOI: 10.1080/00207543.2024.2436126

To link to this article: https://doi.org/10.1080/00207543.2024.2436126

© 2024 The Author(s). Published by Informa UK Limited, trading as Taylor & Francis Group.

Published online: 10 Dec 2024.

Submit your article to this journal

Article views: 2467

View related articles

View Crossmark data

Citing articles: 13 View citing articles

Full Terms & Conditions of access and use can be found at
https://www.tandfonline.com/action/journalInformation?journalCode=tprs20

INTERNATIONAL JOURNAL OF PRODUCTION RESEARCH
2025, VOL. 63, NO. 11, 4114–4136
https://doi.org/10.1080/00207543.2024.2436126

Taylor & Francis
Taylor & Francis Group

OPEN ACCESS Check for updates

Production control with Reinforcement Learning for a matrix-structured production system

L. M. Steinbacher[a,b], T. Wegmann[b] and M. Freitag[a,b]

[a]BIBA – Bremer Institut für Produktion und Logistik GmbH at the University of Bremen, Bremen, Germany ; [b]Faculty of Production Engineering, University of Bremen, Bremen, Germany

ABSTRACT

With increasing product complexities in mass customisation in the automotive industry, the downsides of conventional production concepts like flow production get more pronounced. Their inability to deal with cycle time losses adequately opens up possibilities for new concepts like matrix-structured production (MSP). Due to the immanent dynamics of matrix-structured production, control concept like takt binding or control stands are no longer sufficient to achieve near-optimal performance. The application of Reinforcement Learning (RL) to solve this problem emerged in the recent years. In particular, routing and dispatching tasks have been solved by applying RL. As both tasks influence each other's performance, a combined RL approach is developed. Therefore, a car body construction is simulated to test different modelled Markov processes, algorithms, and rewards. The new approach is validated against common heuristics regarding logistic performance and relevant metrics for operating autonomous guided vehicle fleets. For this, RL systems are designed and compared. The combined approach of production control in terms of dispatching jobs and routing autonomous guided vehicles achieved equivalent performance to heuristics. Still, it excelled in fleet operation metrics, like reduced live or deadlocks.

ARTICLE HISTORY
Received 10 April 2024
Accepted 17 November 2024

KEYWORDS
Production control; Matrix-structured production; Reinforcement Learning; Automotive; Graph Attention Neural Networks

SUSTAINABLE DEVELOPMENT GOALS
SDG 9: Industry, innovation andinfrastructure

1. Introduction

Flow production has been the standard concept in automotive production for 100 years, including body-in-white production (Greschke 2016). This concept is characterised, above all, by its strict takt binding. However, while strict takt binding has many advantages, it also has disadvantages. These include that differences in station cycle times lead to cycle time losses, which reduces output (Neumann and Medbo 2017). These differences in station cycle times can arise from various factors, such as unbalanced workloads between stations, interruptions or delays at specific stations, and the production of multiple product variants within the same system. When station cycle times are not perfectly synchronised, some stations may complete their tasks faster than others, leading to idle times or bottlenecks. Consequently, these inefficiencies accumulate and result in reduced throughput and productivity (Khairai and Khalil 2024).

Various approaches exist to make flow production more flexible and dissolve the strict takt binding, from the simple double cycle, in which individual stations are parallelised, to complex MSP. The latter promises more degrees of freedom and therefore eliminates the strict cycle constraint (Schönemann et al. 2015). Still, the control of these systems has not yet been extensively researched (Nielsen et al. 2023). Those flexible flow production systems have more degrees of freedom, making it more difficult to plan and control. Especially in the case of MSP, it becomes apparent that the system has potential that cannot be exploited with conventional control algorithms, such as fixed routing strategies and static dispatching rules (Renna 2023). In addition, solutions that can do this somewhat better quickly become very complex and barely explainable (Schönemann et al. 2015).

In view of this complexity, the rise of Machine Learning and especially Reinforcement Learning (RL) algorithms promise a solution – not only in Matrix-Structured Production Systems (MSPS) but also in other system layouts (Rai et al. 2021). Therefore, RL was applied to different Production Planning and Control (PPC) tasks. Examples for applied RL are dynamic scheduling (Bär et al. 2020), order dispatching (Waschneck et al. 2018) and routing (Kim et al. 2020). RL is expected to outperform in situations where context awareness and the generalisation of learned patterns lead to better prompt decisions, in comparison to those made using

CONTACT L. M. Steinbacher stb@biba.uni-bremen.de BIBA – Bremer Institut für Produktion und Logistik GmbH at the University of Bremen, Hochschulring 20, Bremen 28359, Germany

© 2024 The Author(s). Published by Informa UK Limited, trading as Taylor & Francis Group.
This is an Open Access article distributed under the terms of the Creative Commons Attribution License (http://creativecommons.org/licenses/by/4.0/), which permits unrestricted use, distribution, and reproduction in any medium, provided the original work is properly cited. The terms on which this article has been published allow the posting of the Accepted Manuscript in a repository by the author(s) or with their consent.

heuristics. As those situations occur predominantly during dispatching and routing these are further put into focus. A more thorough description of those tasks can be found in Section 2.1.

In this context of real-world production environments, tasks related to PPC often influence each other, e.g. defining the next station by dispatching directly influences the following routing task, especially when real-world assumptions like realistic Automated Guided Vehicle (AGV) sizes and safety boundaries are set. Therefore, the interplay of different control tasks with RL is worth investigating, but it has not yet been the focus of literature. Consequently, this paper examines a combination of dispatching and routing with RL. This paper is structured as follows: In Section 2 the background of MSP, production control with RL as well as intralogistics is given. Further, in Section 3, the methodological approach is explained by describing the production environment and its simulation model, as well as the approach of defining the Markov processes in combination with developing different neural network architectures to solve the combined production control tasks. In Section 4, first, the results are described and, secondly, discussed. Lastly, in Section 5 final remarks are given through a conclusion.

2. Background

2.1. Problem description

MSPS have emerged in the context of mass customisation (Koren 2010; Schönemann et al. 2015), as manufacturing mixed products on traditional sequentially arranged production systems can result in starvation or blockages (Bukchin, Dar-El, and Rubinovitz 2002). Moreover, while cellular production systems can mitigate these issues, they are inadequate for handling large production volumes (Hayes and Wheelwright 1979). Over time, several layout concepts have emerged with their specific advantages and disadvantages (Tompkins 2010). Addressing the challenges of mass customisation becomes progressively more difficult as product mixes diverge in their variants. Greschke et al. (2014) proposed a MSP concept that allows individual cycle times of stations, redundant capabilities of stations for multiple work packages within a station. This structure also allows flexible routing of products through the system. An exemplary depiction of these characteristics can be seen in Figure 1.

The underlying problem in MSP lies in the significant challenges associated with production control, meaning the operational control of the production, particularly in dispatching and routing within a highly flexible and dynamic manufacturing environment (Renna 2023). To differentiate between those tasks, the following definitions are followed:

- *Planning* involves creating high-level, aggregated strategies for production over multiple time horizons, aiming to optimise long-term profit by matching supply with demand through effective resource allocation before production activities occur.
- *Scheduling* translates high-level plans into specific tasks, determining start times and machine assignments for each operation to meet demand within resource constraints, ensuring planned production quantities and timelines are achieved.
- *Dispatching* makes real-time decisions to allocate jobs or work-in-progress to machines, focussing on the immediate execution of scheduling decisions to address current production conditions and ensure continuous workflow on the production floor.
- *Routing* determines the paths that transporters will take to move products or materials between machines and stations, optimising travel time and efficiency to support smooth and timely production processes.

Unlike traditional fixed-cycle production systems, MSP requires a sophisticated approach to managing the flow of products and the utilisation of production cells to maintain efficiency and prevent system blockages (Filz et al. 2019).

In this context, dispatching involves the assignment of products to various processing stations in a manner that optimises overall production. The complexity of this task is heightened by the need to accommodate a wide variety of products in MSPS (May et al. 2021), each with different processing requirements, and to ensure that all stations are utilised effectively (Filz et al. 2019). This results in a high potential for deadlocks, where products are unable to move to a free station, and livelocks, where products circulate without making progress (Greschke 2016).

Routing adds another layer of complexity, particularly due to the need to manage high vehicle densities and small batch sizes. The transportation of products must be carefully coordinated to avoid traffic congestion and collisions, which can lead to increased risks of deadlocks and livelocks (Echsler Minguillon and Lanza 2017). Additionally, the need for AGVs to frequently make decisions at intersections and transit points further complicates the routing process (Feldkamp, Bergmann, and Strassburger 2020).

Current research has proposed various strategies to address these challenges, including decentralised agent control, centralised control via control stations, and hybrid approaches that combine rough planning

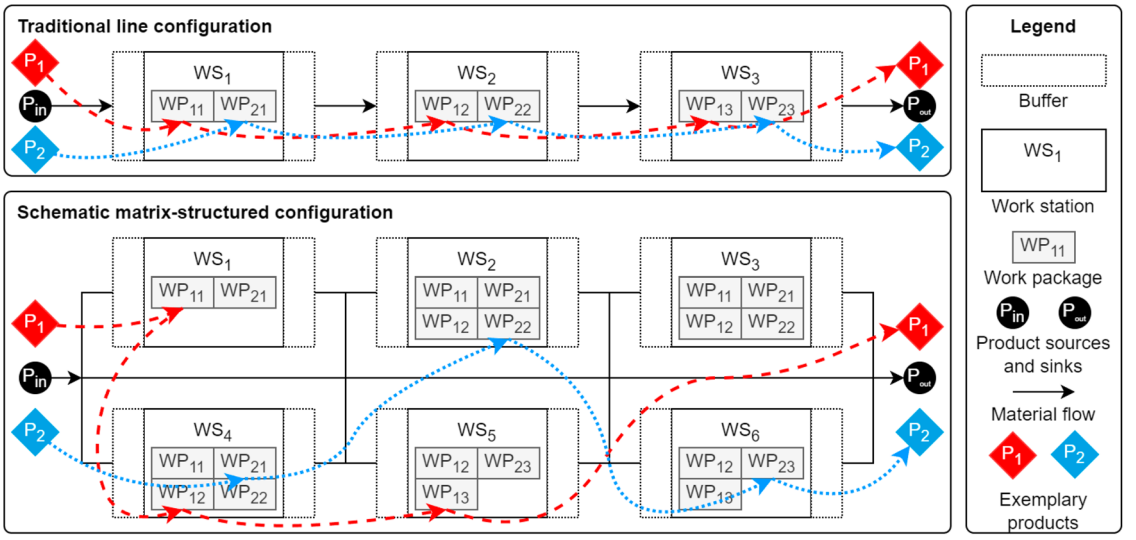


Figure 1. Exemplary depiction of different characteristics between traditional and matrix structured production systems (derived from Greschke et al. 2014).

modules, priority rules (e.g. first-in-first-out and shortest processing time), and RL methods (Echsler Minguillon and Lanza 2017; Greschke 2016). Each of these approaches aims to balance the need for flexibility with the requirement for efficient and reliable production control. However, the inherent complexity of MSPS means that finding optimal solutions remains a significant challenge.

Overall, this work aims to explore and address the complexities and challenges of dispatching and routing in MSP, focussing on optimising AGV utilisation and minimising system inefficiencies.

2.2. Production control in matrix-structured production

In research on MSP, dispatching and control methods are critical topics. Echsler Minguillon and Lanza (2017) present a hybrid approach that adapts priority rules based on the current system state. Their system includes a rough planning module for scheduling, a detailed planning module for dispatching parts to machining centres, and a final module that uses a neural network for optimising priority rules. Initial experiments showed that larger buffer sizes improve the system's performance, but small buffers lead to unsatisfactory schedules.

Hofmann et al. (2018) analyze the benefits of routing and operation flexibility in matrix production compared to traditional production lines. They find that matrix systems offer advantages in throughput time, tardiness, and output, especially when agent coordination is enhanced. Bányai et al. (2019) discuss the optimisation of logistics resources in matrix production, using a cyber-physical system framework. Their mathematical model addresses routing, assignment, and scheduling, with heuristics improving cost-efficiency and environmental impact.

Greschke (2016) explores two primary approaches: decentralised agent control and control via a central station. In decentralised control, agents make rule-based decisions with limited information, which can lead to suboptimal system utilisation and unavoidable blockages, though this method is stable and easily expandable. Centralized control uses three priority rules to achieve full system utilisation, but at the cost of increased complexity (Greschke 2016). Additionally, Greschke identifies the challenge of material provision in matrix production due to multiple assembly locations and the absence of a fixed product sequence.

Further development by Echsler Minguillon (2020) focuses on specific applications of matrix production, where routing of AGVs is based on transport orders. Detailed planning is used to adjust schedules when disruptions occur, with independent agents using RL to decide on the next order. This method reduces buffer size requirements but requires situation-specific solutions due to the lack of generalisation.

Hofmann et al. (2020) present an autonomous production control method based on Deep Q-Learning, which adapts to dynamic system events and compares favorably to static rule-based approaches. This method highlights the benefits of machine learning in handling system variability.

Several recent studies have proposed advanced methods for dispatching and scheduling in matrix production. Zou et al. (2020) introduce a discrete artificial

bee colony algorithm for multi-AGV dispatching, aiming to minimise transportation costs through mixed integer linear programming and advanced heuristics. Their approach shows superior performance compared to existing algorithms. Similarly, Zou, Pan, and Tasgetiren (2021) address multi-compartment AGV scheduling with an iterated greedy algorithm, optimising costs and outperforming other methods in comparative experiments. May et al. (2021) explore decentralised multi-agent production control through economic model bidding, where agents interact to maximise monetary revenues. This decentralised approach outperforms traditional methods for specific control objectives, leveraging RL for intelligent production control.

X.-J. Zhang et al. (2022) also focus on multi-AGV dispatching, proposing an improved iterated greedy algorithm that includes route merging and workshop partition strategies to reduce costs. Their method quickly identifies infeasible solutions and uses effective operators in the local search stage, demonstrating better performance in real-world instances.

The reviewed literature emphasises the ongoing development and refinement of production control methods in MSP. From adapting priority rules to applying RL and other advanced algorithms, these approaches collectively seek to improve system flexibility, throughput, and resource efficiency. The variety of methods highlights the potential for enhanced performance when dynamic system behaviour is effectively managed, particularly through decentralised agent control or machine learning-based approaches. Ultimately, this body of work underscores the critical role of intelligent decision-making in overcoming the challenges posed by MSP environments, such as buffer constraints, routing, and material provision.

2.3. Reinforcement learning

Recent RL advancements have significantly impacted PPC (Panzer and Bender 2022). Deep RL, as applied by Hu et al. (2020), optimises priority rule selection based on the current system state, a concept previously outlined by Echsler Minguillon and Lanza (2017). This approach, demonstrated in a flexible job shop simulation with AGV chaining, focuses on choosing from priority rules rather than direct task allocation. The production time and delay rate are key metrics, serving as rewards for the RL agent. Hu et al. (2020) further enhance this by formulating the AGV scheduling problem as a Markov Decision Process (MDP), detailing state and action representations, reward functions, and an optimal mixed rule policy. Using a novel Deep Q-Network (DQN), the RL agent dynamically selects suitable dispatching rules and AGVs to minimise makespan and delay ratio. Validated through a real-world case study, this method significantly improves scheduling performance, effectively handling high dynamics, complexity, and uncertainty in industrial environments. Other works of utilising rule-based dispatching within dynamic environments came from Luo (2020) and Heger and Voss (2020).

In semiconductor production, Waschneck et al. (2018) implemented RL to train agents in a flexible job shop, hardwired to machines. Each station operates under its own policy, with agents making decisions based on comprehensive observations of station status and lot information. This method effectively reduces late batch proportions, outperforming conventional dispatching rules (Waschneck et al. 2018). Similarly, Kuhnle et al. (2020) developed dispatching agents for adaptive production control, utilising dense and sparse reward functions to excel in dynamic environments with limited domain knowledge. Another research work concerning semiconductor production used DQN to outperform traditional heuristics by meeting time constraints (Altenmüller et al. 2020).

Popper, Yfantis, and Ruskowski (2021) extended multi-agent RL to both station and transport unit control in flexible job shops. The agents assign values to jobs and transport orders, influencing processing and movement decisions. Routing AGVs through complex transportation networks is not included. Despite the lack of a global view and the need for central coordination, this approach surpassed traditional dispatching rules in reducing manufacturing time and product delays (Popper, Yfantis, and Ruskowski 2021). Dittrich and Fohlmeister (2020) also employed central dispatching functionalities. Another addition is the work of C. Zhang et al. (2020). They propose learning Priority Dispatching Rules (PDRs) for job shop scheduling problems (JSSP) using deep RL. They exploit a Graph Neural Network-based scheme to represent JSSP states, resulting in a size-agnostic policy network that generalises well on large-scale instances. Their experiments show that the agent can autonomously learn high-quality PDRs, displaying strong performance against existing PDRs and on larger, unseen instances (C. Zhang et al. 2020).

Contrasting, Bär et al. (2020) assigned RL agents to products instead of machines in a flexible job shop, employing a conveyor belt for movement. The agents' action space, representing 18 possible movements, required learning which actions were feasible from their current positions. Despite varying observations, a unified policy controlled all agents, leading to more effective outcomes (Bär et al. 2020).

Feldkamp, Bergmann, and Strassburger (2020) focussed on RL-controlled AGVs in MSP, with dispatching decisions determining the next station for each AGV. This approach shares similarities with Bär et al. (2020), though it differs in training methods and agent perception. Results indicated that RL dispatching outperformed random methods and was comparable to queue length-based dispatching (Feldkamp, Bergmann, and Strassburger 2020).

In routing tasks within production contexts, RL plays a crucial role. Kamoshida and Kazama (2017) present an AGV-based order picking system where shelves are autonomously moved to order pickers. Unlike traditional routing, their system computes three routes based on different optimality criteria, with the agent's observation focussed on route and node values. The success of this system largely depends on the chosen rewards, showing potential for outperforming random route selection.

Kim et al. (2020) introduce a grid-based packet sorting system, where sorting and feeding units, controlled by individual agents trained via DQN, work towards ensuring each package reaches the right delivery point while maximising throughput. Agents receive rewards based on package delivery accuracy and system input-output ratio, with penalties for collisions. Despite initial training instability, the system showed reduced collisions over time.

Kool, Van Hoof, and Welling (2018) apply Graph Attention Neural Networks (GAN) and RL to solve complex routing problems, including the travelling salesman problem. Their network, comprising an encoder and a decoder, processes node coordinates and solution state information, enabling efficient node selection. This approach treats nodes uniformly, regardless of the input sequence.

Sartoretti et al. (2019) developed the Probabilistic Reinforcement Inference Mechanisms for Adaptive Learning (PRIMAL) architecture for multi-agent routing in grid-like environments. This system uses a decentralised decision-making process, with each agent receiving information about its surroundings through various tensors. The network includes convolutional and Long Short-Term Memory (LSTM) layers, with training through an Advantage-Actor-Critic algorithm. Agents are rewarded based on movement, collision avoidance, and episode completion. Imitation learning is also employed, though the architecture struggles in complex environments with high obstacle density or agent numbers.

Finally, Damani et al. (2021) enhance the PRIMAL architecture by incorporating additional tensors representing ideal routes and future agent positions. They consider scenarios with ongoing goals and fixed-duration episodes, focussing on throughput improvement. This version omits a separate layer for blocking actions, instead using loss augmentation to assess action validity based on predetermined rules.

These studies collectively demonstrate the efficacy of RL in enhancing production dispatching and routing, offering substantial improvements over traditional methods in various production environments. Nevertheless, as depicted in Table 1, all mentioned publications fall short of addressing both control tasks. Therefore, the following presented approach aims to answer how both dispatching and routing tasks can be solved within an industrial application in the automotive industry.

3. Methods

The following methods are employed to answer how RL can be applied to solve combined dispatching and routing in the body-in-white production in the automotive industry. The simulation of the system in question is crucial as RL algorithms need it to train the underlying agents. With an understanding of the system and its derived simulation model, the actual RL algorithms for routing and dispatching are explained. This includes the agent environment cycle, reward structures, the structure of both states and actions, and finally, neural network architectures. Lastly, the training procedure within the experimental setup is explained.

3.1. Simulation of car body construction

In Subsection 2.1 the underlying production control problem is described in a general manner. This subsection provides a detailed perspective on the dedicated production system which is modelled and simulated to research the proposed combined RL approach. In Figure 2, the process and layout structure is depicted, which is based on the MSP described by Echsler Minguillon (2020). In this setup, front and rear ends of car bodies are produced in a fully automated body-in-white production, meaning joining the car body's frame. The stations are interlinked by AGVs and divided into those performing geometry processes and those performing joining processes. Geometry stations are adding and positioning structural parts, whereas joining stations use a multitude of joining technologies to guarantee the car body's structural integrity. Sometimes the fully-automated joining processes (sometimes also called respot processes) lead to imperfect joins which then need to be reworked at the rework station. Unlike previous setups, components are transported on an order-related basis, where the AGVs remain at the station until the component is processed and returned. This approach aims to address

Table 1. Curated list of dispatching and routing applications with RL.

	Characteristics			
Authors	Algorithms	Network	Control Task	Agent
Hu et al. (2020)	DQN	LNN	Choose the best dispatching rule from a given set	AGV
Echsler Minguillon and Lanza (2017)	–	–	Calculate weights for a given set of dispatching rules	Production System
Waschneck et al. (2018)	DQN	LNN	Choose which waiting product will be processed next	Machine
Kuhnle et al. (2020)	TRPO	LNN	Dispatching jobs to machines	Dispatcher
Bär et al. (2020)	DQN	LNN	Choose which way to take at intersections	Product
Popper, Yfantis, and Ruskowski (2021)	PPO	LNN	Machine: Which waiting product will be processed next; AGV: What next station to drive to	AGV and Machine
Feldkamp, Bergmann, and Strassburger (2020)	DQN	LNN	Choose which next station to drive to	AGV
Damani et al. (2021)	A2C	CNN, LSTM	Determine next direction to reach the target position	AGV
Kamoshida and Kazama (2017)	DQN	LNN	Choose the best route from a given set of suggestions	AGV
Sartoretti et al. (2019)	A2C	CNN, LSTM	Determine next direction to reach the target position	AGV
Kool, Van Hoof, and Welling (2018)	REINFORCE	GAN	Find the next node	Salesman /AGV
Hofmann et al. (2020)	REINFORCE	GAN	Find the next node	Salesman /AGV
Dittrich and Fohlmeister (2020)	DQN	LNN	Choose which machine to assign the next order to	Order & machine agents
Kim et al. (2020)	DQN	LNN	Choose which sorter to send the parcel to	Sorter
This article	*PPO, DQN, A2C, Rainbow*	*LNN, ANN, GAN*	*Choose next station for product & which way to take at intersections*	*AGV*

Note: LNN: linear neural network; CNN: convolutional neural network; GAN: graph-attention neural network.

one of the major routing challenges identified: high traffic density and the inefficiency caused by empty runs. Further specifications of the system are presented within the simulation description.

The geometry stations are designed for specific geometric processes and must be approached in a prescribed sequence. The front and rear ends consist of parts that are added to the respective sub-assembly in a dedicated geometry station. Furthermore, there are two different product families, each containing different processes for joining the car bodies' frames. Each sub-assembly (front and rear end) has distinct geometry stations: six for front ends and five for rear ends. In this simulation, each geometric process is completed within 90 seconds. Additionally, material supply occurs at these stations, necessitating their strategic placement at the periphery of the production area. This arrangement ensures that material supply distances are minimised, enhancing efficiency.

Two product variants are developed for both the front and rear ends. Although the joining sequences in Echsler Minguillon (2020) prescribe a fixed order for all joining operations, which is technically not necessary, the joining sequences in this work offer greater degrees of freedom for production dispatching. This increased flexibility, as illustrated in the process structure in Figure 2, provides agents with a larger search space to find improved dispatching strategies. For example, in variant 1 of the rear end, 11 semi-tubular rivets need to be added to the body frame as a work package after geometry station 1 and before geometry station 4. The process structure of the car frame parts in Figure 2 also includes the number of joining operations in each process. The notation *11 STR* indicates joining a part with eleven semi-tubular rivets. In this simulation, each resistance spot welding operation takes 3.5 seconds, semi-tubular rivets type 1 take 5.5 seconds, semi-tubular rivets type 2 take 6.5 seconds, flow drill screws take 9 seconds, ImpAct operations take 3 seconds, and rework operations take 60 seconds.

The cost of an AGV depends on the requirements placed upon it by the parts' characteristics. The requirements should therefore only be as high as the application demands, especially since a double-digit fleet size is required in this application. Because of the drive-through stations, the car frame parts transported by AGV only need to move forward during regular operation. In addition, the layout of the MSP can be adapted to the space requirements of the vehicles. Therefore, a simple tricycle chassis is sufficient. To avoid collisions, a safety scanner is used that looks ahead and causes the vehicle to stop if objects are detected in the safety zone. The safety area

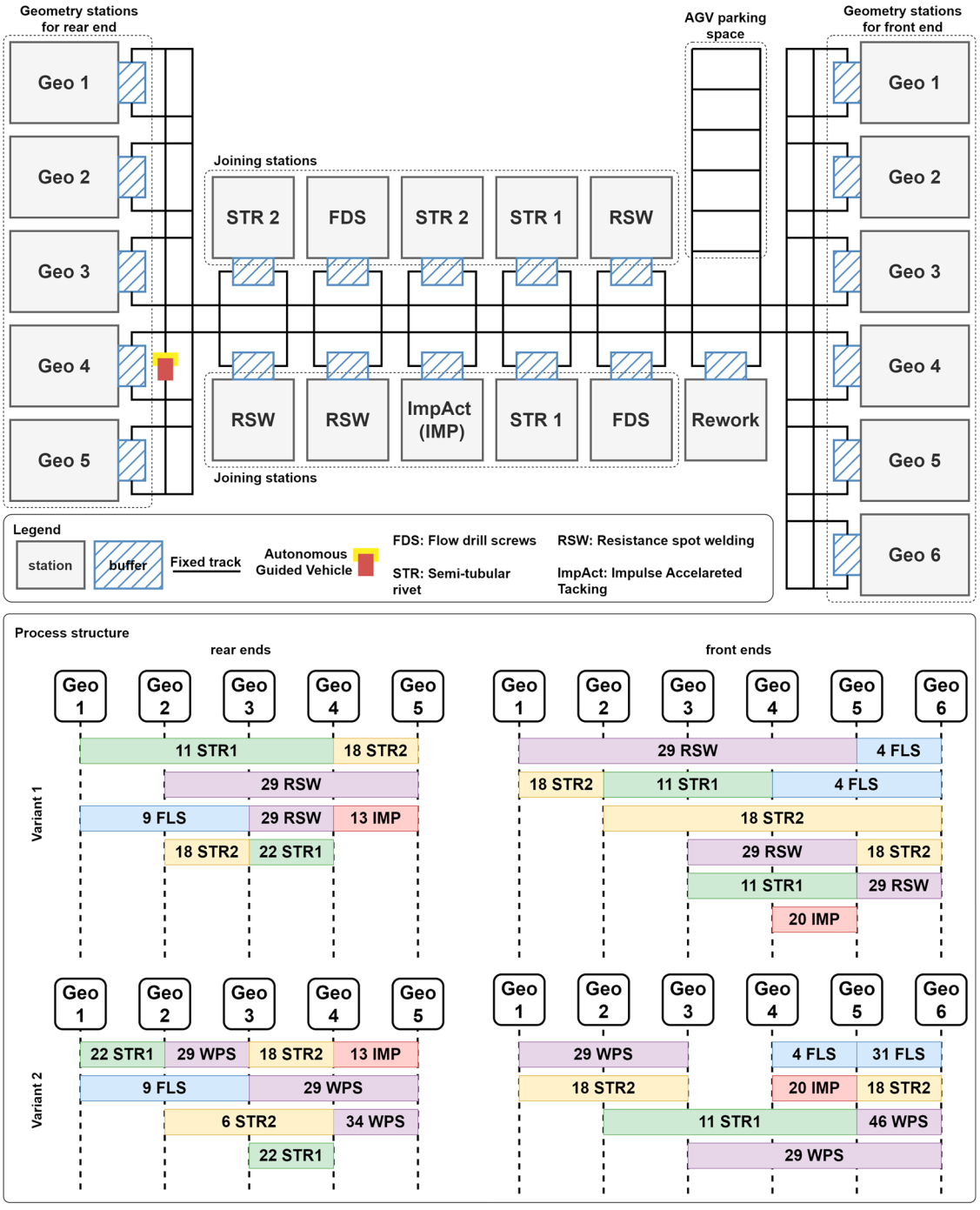


Figure 2. Layout and process structure of the simulated production system.

itself can be parameterised, indicated as yellow marking around the AGV in Figure 2. For this work, we assume a turning radius of two metres. The AGVs should move on fixed tracks. This results in a AGV that is technically simple. The front and rear ends of a car body are relatively large parts. An AGV must therefore be large enough to be able to transport these parts. In this paper, we assume a width of one metre and a length of 1.4 meters. The scan fields must be large enough to avoid collisions with other vehicles. At the same time, it must be possible for vehicles to pass each other and turn in front of another vehicle at an intersection without the scan fields immobilising the vehicles. Regardless of whether a vehicle is controlled by RL or other routing algorithms, it must receive information at decision points about how it should behave. In the case of AGVs, these decision points are all intersections and points where the AGV is scheduled to stop, for example, in transit stations. For intersections, it is

required that if the AGV is to stop, it does not block the intersection for other AGVs.

We chose a simulation time of one hour based on several key considerations. While ramp-up processes do occur, they are very brief (less than 5 minutes) due to the flexibility in the sequence of processes. Furthermore, the lead times are well below 60 minutes, which allows us to observe realistic behaviour, including scenarios where the system runs empty and experiences ramp-up production. This simulation duration ensures that the agent can effectively handle these situations.

Additionally, the velocity of the simulated AGVs is set to 1 meter per second, which provides a realistic reflection of their operational speed. The AGVs follow a decision-making process where, prior to reaching intersections, they request the next direction from a global routing agent. This centralised control ensures efficient routing, avoids deadlocks, and minimises livelocks. Although alternative systems using meshed communication between AGVs could be considered, they offer no significant advantages, and the complexity of updating each AGV's software would be prohibitive.

The presented production system, along with its processes and capabilities, is simulated as a discrete-event simulation with fixed process times. The AGVs are simulated in a physical manner, and travel times are stochastically distributed due to system dynamics, such as blockages and detours. In this context blockages mean at least two AGVs which stop in front of each other and cannot resolve the situation by choosing a detour. Furthermore, 1% of processes include reworking, introducing additional stochastic variability. Combined with the agent-based decision-making through RL for both routing and dispatching, this setup allows the system to handle variability in a realistic manner, validating the effectiveness of the experimental settings, including the chosen simulation time.

3.2. Routing and dispatching with RL

3.2.1. Agent environment cycle

The implementation of RL must be able to map different configurations of agents. In particular, agents that take the perspective of a single vehicle (multi-agent) or agents that make decisions for all AGVs simultaneously and cyclically are possible (single-agent). An agent can take care of dispatching, routing, or both at the same time. It should be possible, for example, for one agent to make the dispatching decision cyclically for all AGVs, and then for one agent per AGV to perform the routing at the intersections. The finalised RL framework is depicted in Figure 3. The system was modelled not as a Markov Game or Markov decision process, but as an agent environment cycle to cover all these cases. The simulation environment takes the role of a special environment agent that maps the consequences of the decisions made by each agent to the states of the other agents. Which agent's turn is next is also determined by the simulation environment.

Analogously to the formal description of an agent environment cycle, the formal elements will be described below and related to the implementation in AnyLogic and Python. The software project was executed through Visual Studio Code and is provided through GitHub (Wegmann 2023), where also all library dependencies are listed.

The simulation environment is defined by a comprehensive framework encompassing various elements (element declaration can be seen in Table A1). The set of possible states, denoted as S, includes all conceivable states within the simulation. The initial state, represented as s_0, is characterised by AGVs starting empty at random positions in the transport network, alongside all stations beginning in an empty state, with the first geometry stations initiating the production of initial parts.

The number of agents, indicated as N, varies depending on the configuration. In a full multi-agent setup, the number of agents is double the number of AGVs, as each AGV is paired with both a dispatching and a routing agent. The set of possible actions for each agent, A_i, is contingent on the control task at hand. For instance, routing for one AGV includes five actions: stop, up, down, left, and right. In contrast, dispatching for an AGV is defined by a number of actions equivalent to the approachable stations and buffer locations. When routing or dispatching for all AGVs simultaneously, this set is replicated for each AGV under control.

The transition function for agents, T_i, varies based on the role. For dispatching, the destination station is incorporated into the AGV's status information and becomes a part of the routing agent's observation. In the case of routing, the executed action is promptly implemented as a driving action by the AGV. The transition function of the environment, P, is delineated by the functionality inherent to the simulation environment itself.

The set of possible rewards and the reward function, R_i, are outlined by the rewards distributed within the simulation environment. These rewards can be customised, differing between routing and dispatching tasks. The set of possible observations, Ω_i, along with the observation function, O_i, remains similar across various agent configurations. To facilitate this, all AGVs and stations transmit their observations to a central controller, which amalgamates these into a comprehensive observation for each agent, thus providing them with insights into the status of other AGVs. When an AGV requires a dispatching

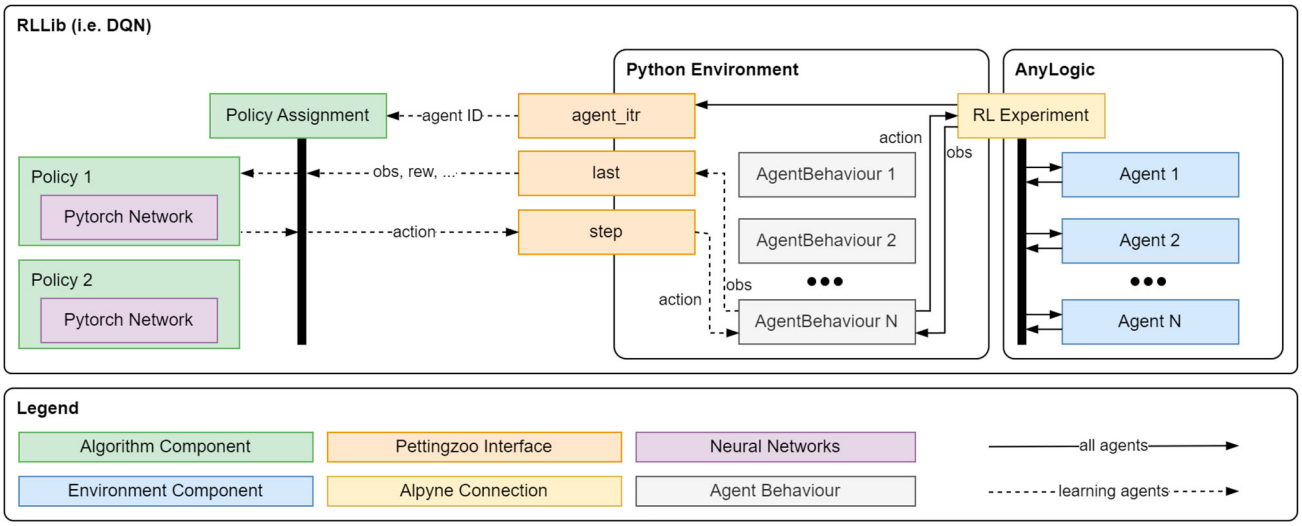


Figure 3. Components of the RL framework.

decision, partial observations from all other AGVs and stations are initially gathered, followed by a decision based on these consolidated observations.

Lastly, the next agent function, ν, also executed by the simulation environment, dictates the sequence of agent involvement. When an AGV reaches a decision point, either for dispatching at a destination station or after a set time or for routing before an intersection, it takes precedence. The environment alternates turns between each agent, ensuring that after a decision is made, the simulation progresses until the next decision point is reached.

3.2.2. Rewards

The following describes the rewards that can be parameterised in the simulation environment, exact values of the final rewards are listed in Table A2. First, the rewards attributed to routing are described. The *reward_target_distance* indicates whether the last reward led the agent closer to its target by path distance (positive reward) or further away from it (negative reward). The *reward_reached_target* is distributed whenever an agent reaches its goal. If an agent reaches a station node that was not its destination, the *reward_wrong_target* is posted. If an AGV is in a blockage that can only be cleared by removing an AGV, it receives the *reward_removed_for_block*. The AGV should also receive feedback if it does not move. This is done by the *reward_invalid*, which is issued whenever the AGV is at a node that is not the target node and chooses not to move or tries to move in an invalid direction. This reward has caused problems in early experiments. A low value causes AGVs not to move, since they still collide frequently with other AGVs in the beginning, and this is not compensated by the very rare reward for reaching a destination. A high value causes the vehicles to never stop, but this is part of the desired behaviour to avoid collisions. Therefore, the same reward is not given every time a vehicle stops, but long stops are penalised the longer they last. In order for an AGV to be able to estimate the impact of a standstill, the number of previous actions that prolonged the standstill is part of the observation. It could be observed that it is quickly learned to choose certain driving directions on certain sections of the route. To encourage this behaviour without giving up the freedoms of the bidirectional transport network, a *reward_direction* is introduced that leads to a negative feedback for one driving direction at a time. This gives the agent a default for which driving direction is more attractive, but it can also use the other direction if needed. In addition, a reward is generated to optimise the blocking behaviour of the AGV. The *reward_pass* is issued whenever an AGV stops at a node that is currently being approached by another AGV. So far, livelocks have not been taken into account. The simulation detects whether a livelock is present by comparing how long the AGV took to complete a path and how long it should have taken. If the AGV takes significantly longer for a path than it should have, a livelock is assumed. A livelock leads to the reward *reward_duration*. The *reward_duration* is the only reward that is issued for routing and dispatching.

Especially relevant for dispatching are the rewards *reward_declined_in_station* and *reward_accepted_in_station*, which are issued whenever an AGV is processed or loaded in a station or rejected by the station. In addition, process-related rewards can be issued for geometry processes (*reward_geo_operation*), completion processes (*reward_respot_operation*) and rework processes (*reward_rework_operation*). The *reward_part_completed*

signals that a part has been completed at the last geometry station.

For dispatching, the agent strives to move to as many stations as possible in as few steps as possible where it is accepted or processes are executed. However, especially for dispatching, the number of steps is not equivalent to the time elapsed or the distance travelled, since in multi-agent setups the agent is only invoked upon arrival at its destination, so the agent has no incentive to select stations that are as close as possible. For this reason, the *reward_reduce* is introduced, which causes a negative reward for each routing step that the AGV takes. This reduces the positive reward for arriving at the correct destination for dispatching.

During the experiments, it was noticed that dispatching has another problem resulting from the two families of components. An empty AGV can decide which first geometry station to go to in order to pick up a new part. But if the agent predicts higher rewards for one of the two part families, the vehicles will only drive to that part family. If by chance the other part is picked up after all, it will lead the agent to the geometry stations of the wrong part family at the next stations where it will be rejected. The system therefore tends to one of the two part families in an unstable manner and does not learn to process them simultaneously. Two solutions were devised for this. First, a *reward_balance* was introduced, which, once the difference between front and rear ends in the system was greater than one, was output in proportion to that difference. In addition, the possibility was created to divide the vehicles into two fleets, so that separate policies could be learned for dispatching front and rear ends.

3.2.3. States and actions

The observation for the AGVs has the form (n_{agvs}, $n_{features}$) and the observation for the stations has the form ($n_{stations}$, $n_{features}$). n_{agvs} describes the fleet size, $n_{stations}$ the number of stations. The features are composed of different parts. For AGVs, information about the current position of the vehicle, possible directions of travel, and loading status are encoded. In a multi-agent behaviour, the output of a network has the form ($n_{actions}$). $n_{actions}$ differs between dispatching and routing. For routing, five actions are possible; for dispatching, the number of possible actions is equal to the sum of $n_{stations}$ and the number of possible buffer locations. A detailed description is provided in Table 2.

Table 2. Observations of AGVs and stations.

Feature	Length	Content
AGV Observation		
AGV State	3	Is the AGV active, is it moving and if not, for how long has it been standing?
Position	14	coordinate of last node, distance to last node, coordinate of next node, distance to next node, coordinates of the (up to 4) nodes that can be reached from the next node
Target	2	coordinate of target node
System State	2	count of front and back ends within the system
Part State	35	part family and variant, next geometry process, state of respot processes
Station Observation		
Position	2	coordinate of the network node that resembles the agv transfer station
Station State	7	one-hot-encoded state of the station

3.2.4. Neural network architecture

The neural network architectures developed for multi-agent behaviour share common input structures and address different decision-making processes, namely routing and dispatching. The observations for each AGV and station are key inputs to these networks, facilitating informed decision-making.

The observation for the AGVs has the form (n_{agvs}, $n_{features}$), where n_{agvs} represents the fleet size, and $n_{features}$ denotes the number of features in the observation. Similarly, the observation for the stations is structured as ($n_{stations}$, $n_{features}$), with $n_{stations}$ indicating the number of stations. The output for each network is ($n_{actions}$), where $n_{actions}$ varies between routing and dispatching tasks. For routing, five actions are possible, while for dispatching, the number of actions equals the sum of the number of stations and the possible buffer slots.

Routing focuses on determining the path an AGV should take, often requiring an understanding of spatial relationships and potential collisions. Dispatching, on the other hand, involves deciding which task or destination an AGV should be assigned to, necessitating a comprehensive evaluation of task priorities and resource availability.

The first network for multi-agent behaviour enhances the input data with Fourier Features as described by Tancik et al. (2020). The Linear Neural Network architecture processes the data of the deciding AGV through four linear layers, with additional data from other AGVs and stations integrated and processed separately (see Figure 4):

- *Observation Input*: (n_{agvs}, $n_{features}$), ($n_{stations}$, $n_{features}$)
- *Output*: ($n_{actions}$)
- *Fourier Features*: Applied to all inputs
- *Layer Type*: Multiple linear layers
- *Key Feature*: Aggregation and separate processing of AGV and station data, ensuring consideration of relationships to the deciding AGV

The second multi-agent network utilises an attention mechanism. Fourier Features are initially added, followed by two linear layers for each entity. The remaining AGVs

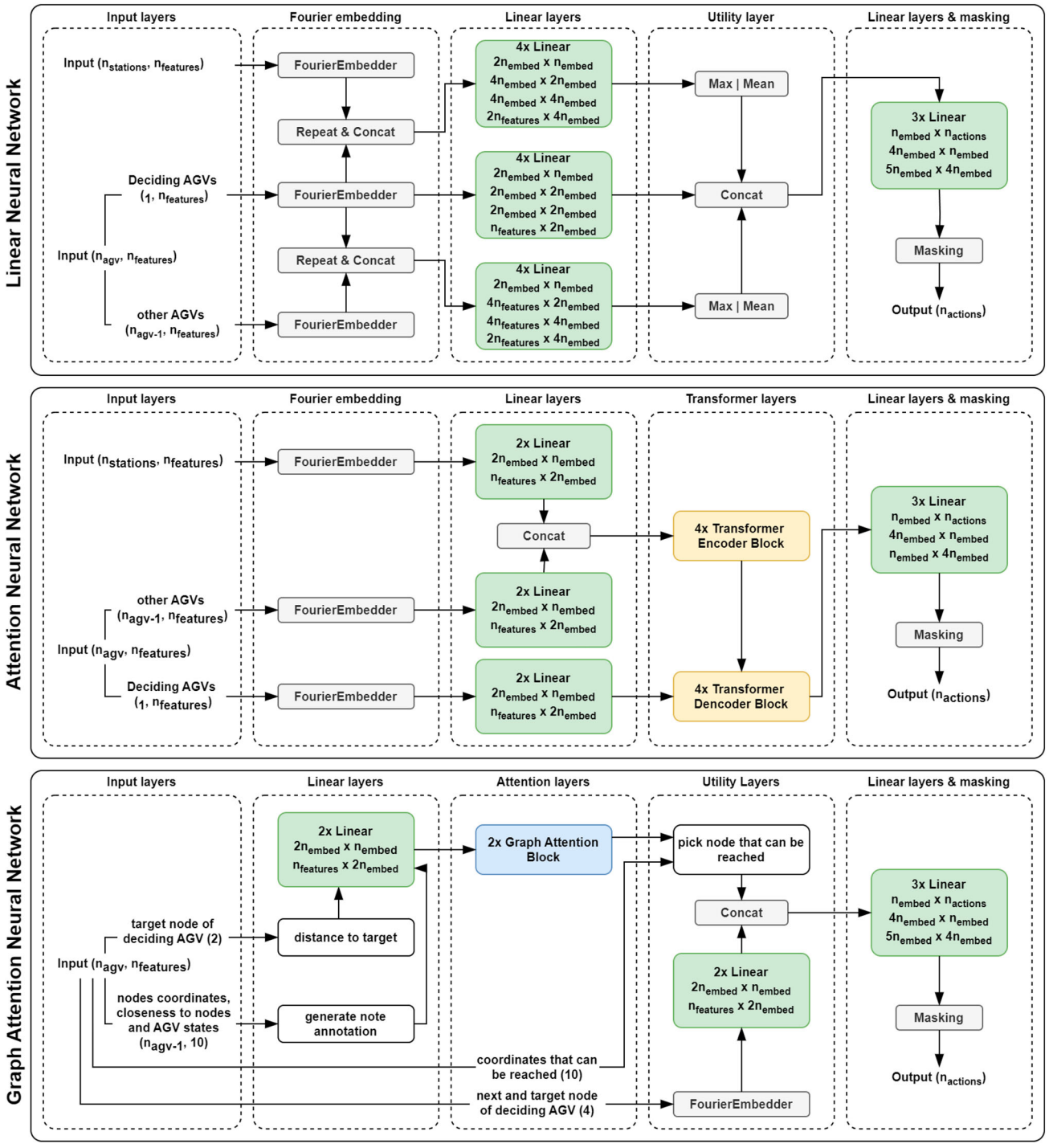


Figure 4. Overview of neural network architectures.

and stations undergo a self-attention layer before interacting with the deciding AGV through a Transformer Decoder Block. This Attention Neural Network architecture is inspired by the original Transformer model proposed by Vaswani et al. (2017) (see Figure 4):

- *Observation Input*: $(n_{agvs}, n_{features})$, $(n_{stations}, n_{features})$
- *Output*: $(n_{actions})$
- *Fourier Features*: Applied to all inputs
- *Layer Type*: Linear layers, Self-Attention, Transformer Decoder Block
- *Key Feature*: Interactions between AGVs and stations are mediated by self-attention and transformer mechanisms

The third network adapts the structure used by Sartoretti et al. (2019), replacing convolutional layers with graph

attention layers to better suit the transport network environment. Observations are converted into a graph format where each node represents an observation coordinate, annotated with binary values indicating AGV presence. This Graph Attention Neural Network approach follows the method by Kool, Van Hoof, and Welling (2018), utilising graph attention layers to capture the relationships between nodes (see Figure 4):

- *Observation Input*: Graph format derived from (n_{agvs}, $n_{features}$)
- *Output*: ($n_{actions}$)
- *Layer Type*: Linear layers, Graph Attention blocks
- *Key Feature*: Graph-based representation allowing for a detailed assessment of node attractiveness for the deciding AGV

Each network architecture was designed to address specific challenges within the matrix production domain, leveraging different mechanisms and structures to optimise decision-making processes and improve overall system efficiency.

3.3. Training procedure and experimental setup

As stated previously, RL training schemes in published research fall short of explaining how specific results came about and why certain training procedures were employed. This is often caused by a lack of resources to run an extensive parameter search. Consequently, the alternative approach is defined by a subset of parameters, which are tested and improved iteratively. Accordingly, this subsection outlines this iterative approach. It is subdivided by training routing and dispatching agents separately, as later shown, training the agents for routing and dispatching together is not viable because training processes are too unstable for realising convergent behaviour. The explored hyperparameters are listed in Table 3.

Table 3. Hyperparameters for routing and dispatching tasks.

Task	Hyperparameter	Values
Routing (*R*)	Architecture (*R-A*)	Linear NN (*R-A1*)
		Attention NN (*R-A2*)
		Graph NN (*R-A3*)
	Gamma (*R-G*)	0.9 (*R-G1*)
		0.98 (*R-G2*)
		0.999 (*R-G3*)
	Learning Rate (*R-LR*)	$1e^{-3}$ (*R-LR1*)
		$1e^{-4}$ (*R-LR2*)
	Definition Epoch (*R-E*)	no new agent (*R-E1*)
		on block (*R-E2*)
		on block or at target (*R-E3*)
	Reward Shaping (*R-RS*)	standard rewards (*R-RS1*)
		reward_direction (*R-RS2*)
		reward_direction & rewards_target_distance (*R-RS3*)
		stronger rewards block (*R-RS4*)
Dispatching (*D*)	Agent and Architecture (*D-A*)	Multi-agent in PPO with masking (linear)(*D-A1*)
		Multi-agent in DQN (linear)(*D-A2*)
		Multi-agent in PPO with masking (Attention)(*D-A3*)
	Gamma (*D-G*)	0.9 (*D-G1*)
		0.98 (*D-G2*)
	Learning Rate (*D-L*)	$1e^{-3}$ (*D-L1*)
		$1e^{-4}$ (*D-L2*)
		$1e^{-5}$ (*D-L3*)

For routing, preliminary experiments were conducted in a much smaller environment to determine whether multi-agent or single-agent systems should be used, and which algorithm should be used to train them. A multi-agent system was chosen for all applications. Furthermore, preliminary a DQN algorithm was selected for training routing. For dispatching the Proximal Policy Optimization (PPO) with masking outputs was the best option in terms of performance and computing time. Other algorithms like Advantage Actor Critic (A2C) or Rainbow algorithms did not performed as well as the chosen ones. The further definition of hyperparameters is based on these initial choices. A full hyperparameter permutation was avoided to deal with the limitations of computational resources, so this sequential approach with best fit hyperparameters was chosen. While training the routing agent, dispatching to viable stations is done randomly, as more sophisticated methods like the shortest waiting time heuristics are not feasible in this dynamic system. The first design decision concerns the network architecture and evaluating which architecture yields the most targets reached compared to the number of blockages. With the chosen Graph Neural Network architecture, the learning rate and the gamma value variations are then performed. Next, how long an epoch lasts from the agent's point of view is varied. An epoch may run for the entire contiguous simulation time (one hour) or end when a blockage occurs or a target is reached. In both situations, the further course is random (selecting the next destination or repositioning after a blockage) and thus difficult for the routing agent to estimate. Finally, the rewards are adjusted and tested to get as close as possible to the desired routing behaviour.

The dispatching agent is also trained in isolation first. During training, routing is done by a baseline routing algorithm with collision avoidance (Möhring et al. 2004). First, it is tested whether all AGVs should be controlled by a central agent or by one agent each, and which algorithm best suits this. Again, the learning rate and gamma value are then varied, after training both agent types separatly.

The combined training procedure involved further training the pre-trained dispatching and routing policies together, starting with fleet sizes between eight and twelve vehicles and then increasing to up to sixteen vehicles. This process was noticeably more unstable compared to previous attempts, likely due to the challenges of hierarchical RL where policy changes in one agent create altered conditions for other policies.

The previously explained training procedures produce results, which are discussed in Section 4. After finding suitable hyperparameters for both routing and dispatching, the trained agents are combined to be trained together. The experiment runs are executed with three different random initializations each time. The production program for the final evaluation is generated randomly. The first product variant of both rear ends and front ends is chosen with a probability of 70% and the second with 30%.

All experiments are executed on a system with 8 GB RAM, an Intel Core-i5-4210U CPU with 1.7 GHz and a Nvidia GeForce 840M graphics card.

4. Evaluation

4.1. Results

In evaluating the performance of various approaches to routing and dispatching tasks for AGVs, several key performance indicators (KPIs) were analysed to determine their desirability and effectiveness. The primary KPIs included the number of blockages, reward values, targets reached, lead time, and products completed. A low number of blockages is highly desirable as it indicates smoother operation and fewer interruptions in the network, leading to more efficient routing and dispatching. High reward values, which show convergence, are crucial as they reflect stable and effective learning processes within the system. The number of targets reached should be maximised, signifying the AGVs' efficiency in completing assigned tasks and effective dispatching. Similarly, a low lead time is critical as it indicates faster and more efficient operations, reducing delays in the production process. Finally, a high number of products completed is an essential KPI, reflecting the overall productivity and success of the routing and dispatching strategies. The results are shown in graphs in this section that depict the area between the maximum and minimum values of the given KPIs resulting from 10 repeated training runs.

4.1.1. Routing

The results are discussed in the sequence of changed parameters: The first step is to test which neural network architecture is to be used. This involves looking at how many targets are reached during the trial period and how many AGVs must be repositioned within the network due to blockage situations. Figure 5(a) shows that the Graph Attention Neural Network learns the fastest to achieve the most reached targets with relatively fewer blockages than a linear network. By far, the fewest blockages occur in the Attention Neural Network, but this network also achieves almost no targets and seems to prioritise avoiding blockages disproportionately. Consequently, the Graph Attention Network is used for the following experiments.

Afterward, the values for *gamma* and the *learning rate* are varied with the described standard episode reward. A high value for gamma results in worse reward outcomes in the experiments performed. In particular the avoidance of blockages works best with a gamma value of 0.9 (see Figure 5(b)). A smaller gamma value gives more weight to short-term and direct consequences of actions.

Concerning the *learning rate*, it was investigated whether a reduction of the learning rate and, thus, a smaller, more gradual optimisation of the network leads to better results. The results show that the training takes place more slowly, and the learned behaviour of preventing blockages manifests much later during the training runs than with the higher learning rate (see Figure 5(c)). Since this effect was already evident in the first trial run, further trial runs with blockages were abandoned. The choice of how to terminate an episode differs reward-wise between ending it prematurely or running it until the end of the simulation. Ending them prematurely leads to a similar negative influence on the achieved reward but positively affects the avoidance ofblockages. Since blockages occur relatively often, episodes are terminated for the following attempts in case of a blockage (see Figure 6(a)).

Next, the rewards were adjusted. The *reward_target_distance* and the *reward_direction* were tested, and the *reward_block* was increased. The *reward_direction* increases the number of reached targets, so it is kept. The *reward_target_distance* should accelerate the training process by rewarding an approach to the target. Approaching the target is rewarded, even if the agent has not yet learned to reach it. This could not be proven in the experiments. An increase in *reward_block* did not affect the blocking behaviour but led to fewer goals being reached (see Figure 6(b)).

During the training, the system was observed concerning the reward and other behaviours. In many situations, deadlocks occurred that could have been easily avoided. Thus, the algorithm does not seem to react sufficiently to the presence of other vehicles. To address this, the *reward_pass* was introduced. This reward occurs whenever an AGV stops to let another AGV pass.

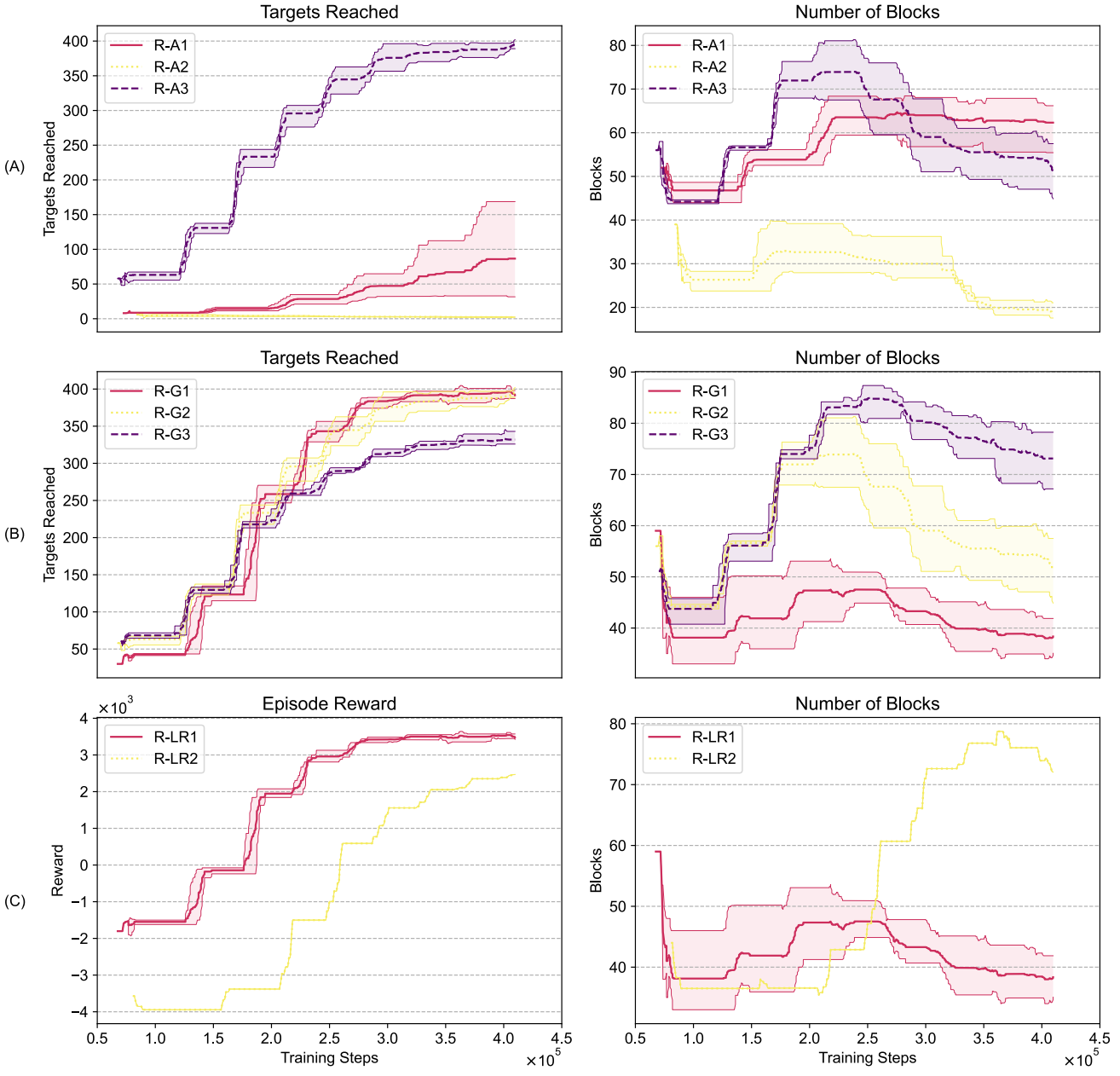


Figure 5. Subset of training procedures, varying the neural network architecture, gamma, and learning rate values.

Although the test results with the new reward do not show a significant improvement, it was observed that AGVs stopped letting other AGVs pass when observing the trained system. Therefore, it was tested again whether the reward in combination with a higher value for *reward_block* leads to an improvement. A significant decrease in collisions can be observed in Figure 6(c) with hardly any change in the number of targets reached.

4.1.2. Dispatching

By testing different learning algorithms and neural network architectures, the hyperparameters of the dispatching solutions are explored (see Figure 7(a)). First, which learning algorithm and network are suitable for control dispatching is determined. Using the Proximal Policy Optimization (PPO) algorithm, it was tested whether the single-agent behaviour or the multi-agent behaviour leads to better results. Again, the multi-agent behaviour, in which the next target is always set for an agent, is shown to learn faster. The single-agent behaviour requires that a goal is periodically specified for all AGVs. Especially in the early phases of training, this can lead to AGVs frequently being assigned new targets before reaching their previous target. In this respect, it can be explained that learning in single-agent behaviour is very slow. Thus, single-agent setups were discarded. The PPO algorithm was masked in all experiments so that the

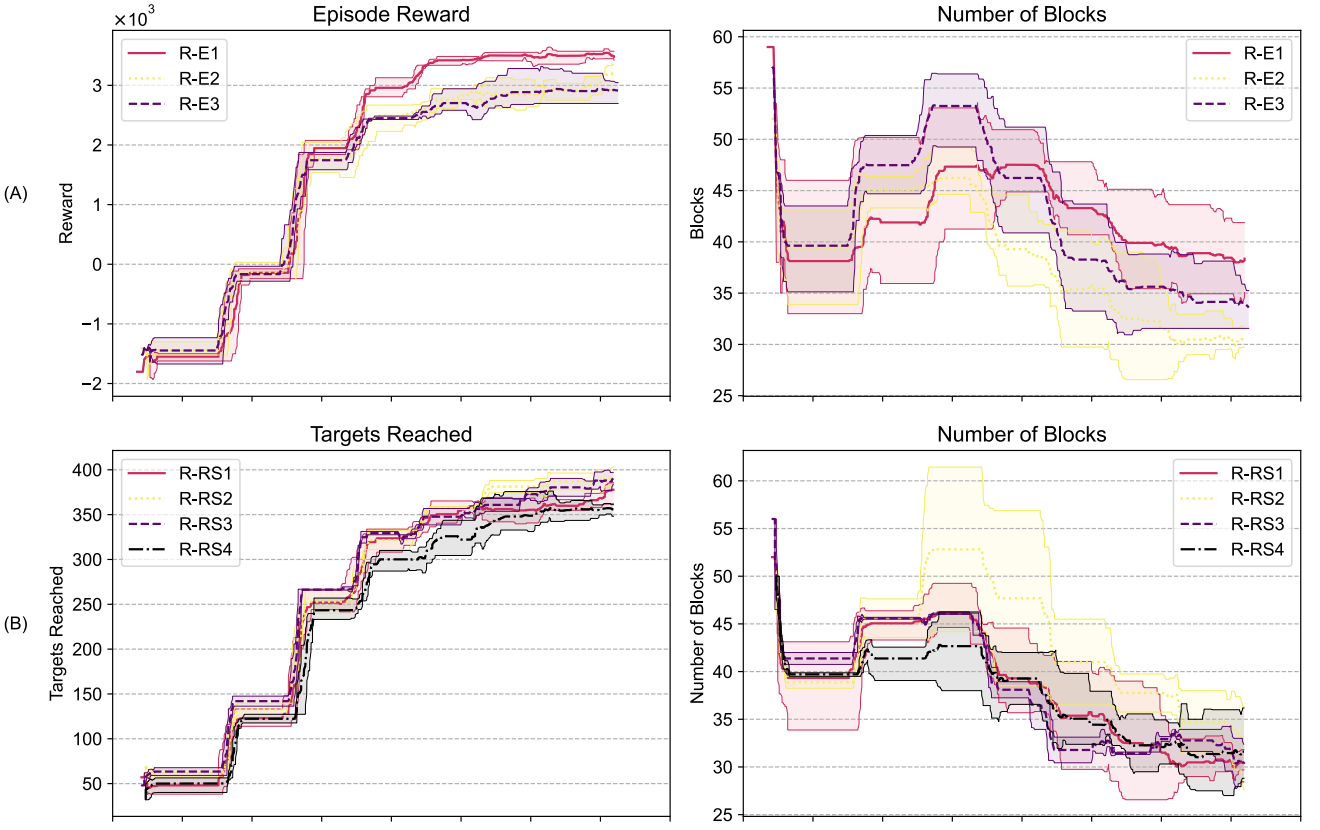


Figure 6. Subset of training procedures, varying the definition of episodes and rewards.

network could only select stations that did not reject the AGV. This includes the buffers. This masking does not work for the DQN algorithm but the routing experiments showed that the DQN algorithm learns faster. The DQN algorithm starts at a significantly worse initial position due to the lack of masking. It learns almost as fast as the masked PPO algorithm. Nevertheless, the PPO algorithm is selected due to the shorter training time. Only the network of linear layers and the network of attention layers are available as networks. The graph-attention network used for routing is not used because dispatching requires the global view of the production. The advantage of the graph-attention network is that it is precisely based on considering local contexts. Attention and linear layers provide comparable results, but the results of the attention network are slightly worse with longer computation time.

The values for *gamma* and the *learning rate* were determined in the same way as for the routing. Here, too, the lower gamma value is better. Against the background that dispatching usually has to be decided on the basis of the situation and that there are few correlations that exist over long periods of time, this is understandable (see Figure 7(b)). Unlike routing, it can be shown that the best results are achieved with a slightly lower learning rate (see Figure 7(b)).

4.1.3. Combined routing and dispatching

Finally, the learned dispatching and the learned routing policies were brought together. Both were first pre-trained with the hyperparameters determined in the previous section. When training the routing, random dispatching was chosen for the pre-training. This ensures, at least initially, that all targets are learned equally. Early use of meaningful dispatching may result in many vehicles going to the same destination at the same time. For routing, this can lead to learning not to approach this area at all if too many negative experiences have been gained there due to collisions and the high traffic density. Random dispatching avoids this. Dispatching first trains with conventional routing. This is more time efficient since dispatching is called much less often in joint training than routing. The learned policies were further trained together.

The trained agents were compared with conventional approaches to routing and dispatching. For this purpose, each vehicle was simulated ten times for one hour, with different fleet sizes of eight and twenty vehicles. In addition, the learned algorithms were combined with the conventional algorithms. Figure 8 presents the results. The system's overall performance can best be read from the products produced. The conventional approaches to routing and dispatching produce the worst result in

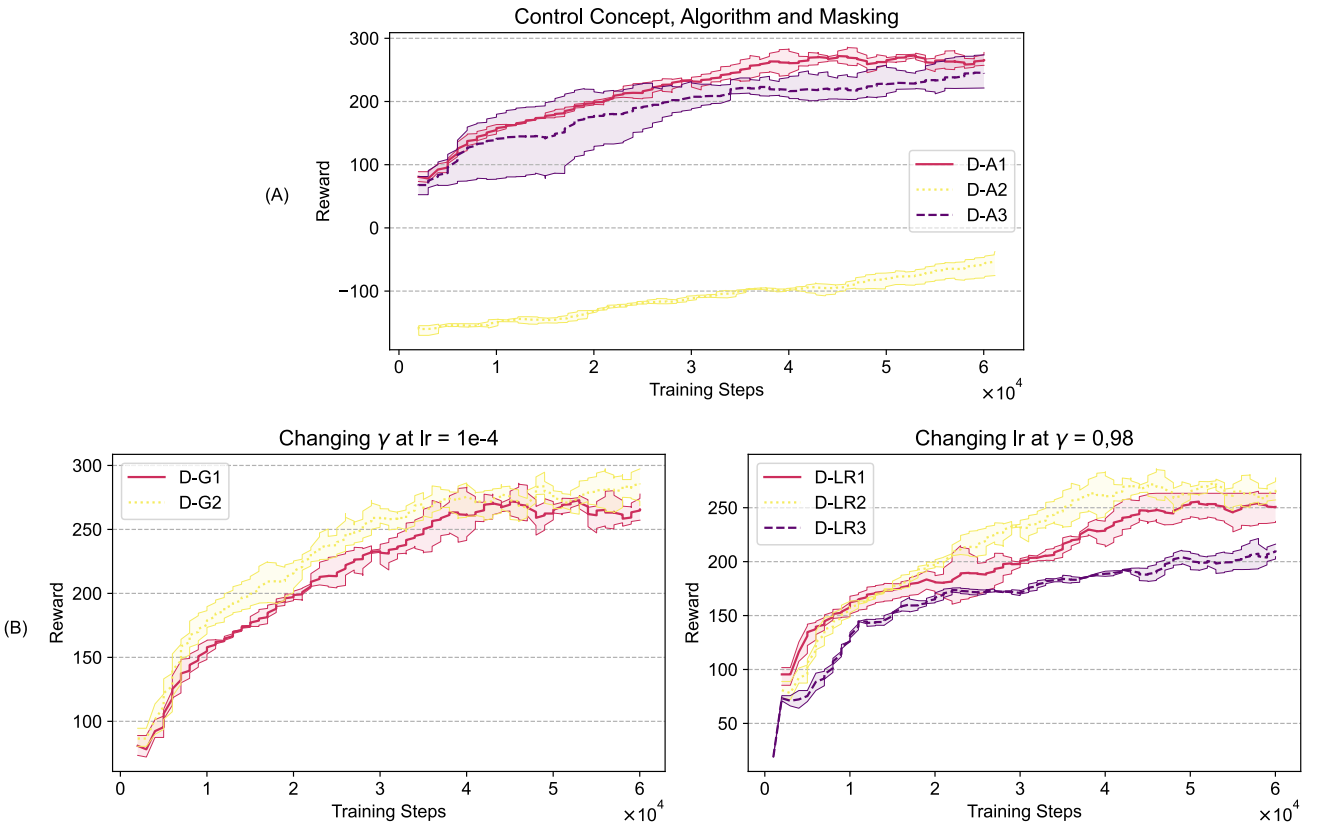


Figure 7. Subset of training procedures, exploring masking concepts, agent configuration as well as learning rate and gamma values.

comparison. Not only are the fewest products produced within an hour with these approaches, but the number of vehicles removed due to blockages is also highest with these methods. Although conventional routing is supposed to rule out blockage situations, dynamic effects such as the formation of queues are not taken into account in route planning, so deadlocks can occur. In addition, conventional routing means that vehicles do not have a preference for one lane, so they often drive in one direction on both lanes and oncoming vehicles do not pass or have to wait for a long time. The number of targets reached is also the lowest for all fleet sizes. Only by setting a fixed driving direction can both the blocking behaviour and the number of completed transport orders be significantly improved.

Using a learned dispatching policy with conventional routing and a specified direction of travel does not change the number of products manufactured significantly for all fleet sizes. The same is observed for blockage situations and lead times.

Once learned routing is combined with learned dispatching, most products are manufactured for small fleet sizes. With increasing fleet sizes, the number of blockages increases again and develops a plateau for larger fleets with more than 16 AGVs, the number of blockages no longer increases, but the number of products completed decreases more than in the other approaches. Lastly, the lead time of combined trained approaches are the lowest upon all combination with the exception of an AGV fleet size of 18.

Finally, the combination of learned routing policy and conventional dispatching was tested. Here, the results create a mixed picture. Regarding number of blockages, this combination strictly dominates all other combinations. When the KPIs number of reached targets and number of products completed these combined routing and dispatching policies are worse than the other approaches in small fleet sizes but better with larger fleet sizes. In regard of lead times this combination is comparable with conventional control algorithms.

4.2. Discussion

4.2.1. Training procedures for dispatching and routing

All experiments have shown that training a policy that makes decisions for one agent at a time at the right moment (multi-agent behaviour) is more favorable than making decisions for all AGVs simultaneously and periodically through a policy with higher-dimensional action space (single-agent behaviour). The reason is probably

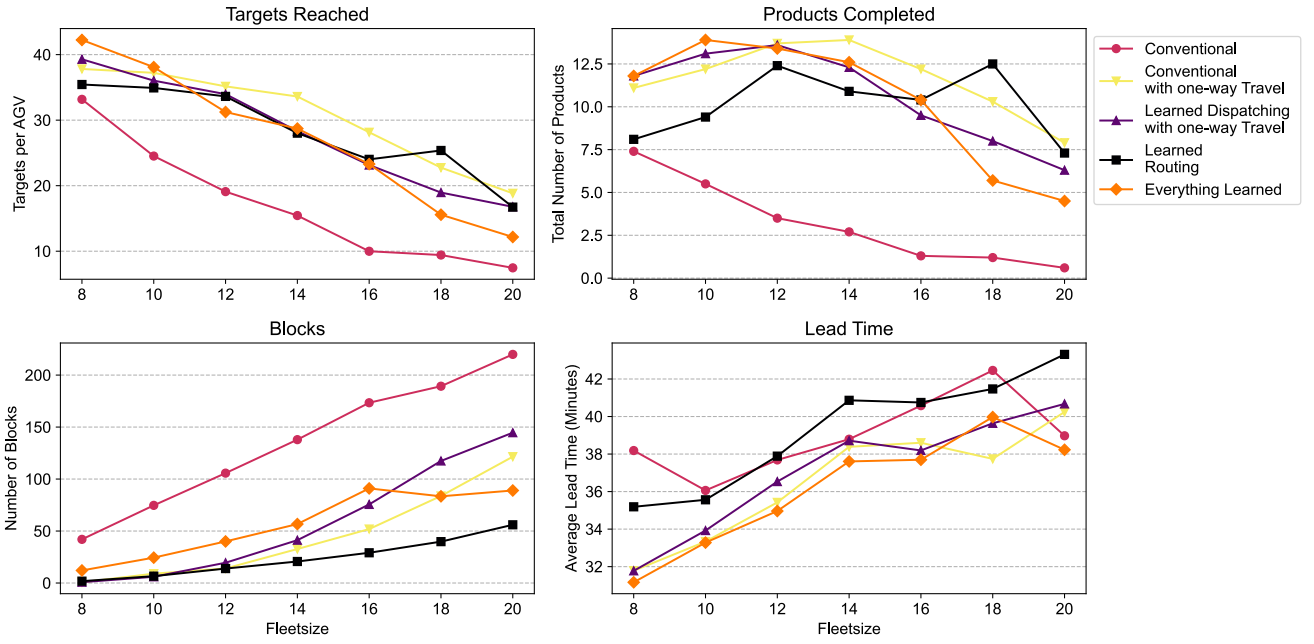


Figure 8. Subset of experiments exploring the combined training of dispatching and routing agents.

that the environment issues only one reward for a higher-dimensional action space. It is difficult to deduce from this which of the actions were good and which were bad. Nevertheless, the concept would have its advantages. For example, in the case of two AGVs approaching an intersection, one AGV could deliberately wait and the other pass since both decisions could be related to each other. The attention mechanism, in particular, would lend itself to this. For future projects, it would, therefore, be interesting to develop algorithms that can train a policy with multidimensional action space through multiple reward signals. Regarding the choice of algorithms, it can be said that different algorithms can provide benefits for different tasks. The DQN algorithm was the most efficient routing option. For dispatching, on the other hand, the PPO algorithm was used because the masking of the invalid actions gave an advantage that could not be matched by the DQN algorithm in the training period considered. It is also possible that the A2C or Rainbow algorithms are better suited for other tasks due to their individual advantages, although they did not prevail for this work. It has been shown that the learning rate and gamma value can greatly impact the training outcome. The amount of individual rewards seems to be a less important factor. They rather serve to change the learned behaviour in the desired direction.

For routing, it was found that a Graph Neural Network-based architecture is critical to the success of training. More generally, it can be said that a well-designed architecture is important for complex RL problems. Moreover, for coordinate-based problems, it seems to be a good choice to use Fourier features. The choice and representation of features, i.e. the values applied to the network's input layer, have been identified as crucial factors for dispatching. This finding also applies to other dispatching problems.

4.2.2. Combined dispatching and routing with RL

The results from Subsection 4.1.3 clearly demonstrate that conventional dispatching and routing algorithms, specifically random dispatching and conflict-avoiding routing as described by Möhring et al. (2004), are insufficient for managing the complexity of MSPS under realistic conditions. When the travel direction is unrestricted, the transport network's complexity is particularly high, and conventional algorithms fail to load the system effectively. This observation underscores the inherent limitation of conventional algorithms in controlling the complexity of MSPS.

Motivated by the potential of RL to handle complex systems, we explored learned strategies for both dispatching and routing. The learned algorithms showed promise in coping with system complexity. The combined training for dispatching and routing agents, even with a maximum of 16 AGVs, demonstrated effective blockage-avoiding behaviour that generalised well to larger AGV fleets used in some experiment runs with 18 and 20 AGVs.

When the complexity is reduced by specifying directional paths, conventional algorithms can better manage the system, resulting in increased targets reached and products completed. However, this reduction in complexity comes at the cost of routing flexibility, which is critical for large AGV fleets in confined spaces where

overtaking maneuvers and efficient queue formations are necessary.

Our results indicate that using a learned dispatching policy with conventional routing and a specified direction of travel does not significantly change the number of products manufactured across different fleet sizes. This consistency is attributed to the learned dispatching policy's ability to mitigate the adverse effects of random dispatching, ensuring that all targets are initially learned equally and preventing premature convergence to suboptimal dispatching strategies.

The impact of routing appears to be more pronounced than dispatching regarding targets reached, products completed, and lead time. The combined approach of learned dispatching and routing exhibited superior performance for smaller fleet sizes but faced challenges with larger fleets. As the fleet size increased, the number of blockages also increased, leading to a plateau in performance for fleets with more than 16 AGVs. Despite this, the lead time for combined trained approaches remained the lowest across all combinations, except for an AGV fleet size of 18, highlighting the effectiveness of learned routing in managing system dynamics.

The combination of learned routing policy and conventional dispatching yielded mixed results. It dominated all other combinations in terms of reducing blockages, yet its performance in reaching targets and completing products was inferior for smaller fleet sizes but improved for larger ones. This suggests that learned routing's adaptability to dynamic traffic conditions is beneficial, but conventional dispatching's limitations become apparent as fleet size increases.

The observation that fewer products are manufactured as fleet size increases, with no approach reversing this trend, suggests that the production setup cannot be fully utilised due to increasing blockages and decreasing fulfilled transport orders per AGV. This aligns with the notion that hierarchical RL struggles to maintain performance for infrequently invoked orders by the worker-agent, highlighting an area for future research.

The integration of conventional algorithms with learned algorithms revealed that learned dispatching improves output with both learned and conventional routing but does not enhance lead times without additional structural information at the dispatching level. Priority rules, such as dispatching based on the shortest route to the destination, could potentially address this gap.

Overall, our findings indicate that while RL offers a promising approach to controlling complex production systems, the intricacies of MSPS present unique challenges. The simulation environment created, along with the Python adapter for Gym, provides a robust foundation for exploring these challenges further with alternative algorithms or network structures. This research contributes to the broader field of hierarchical multi-agent RL, emphasising the need for continued exploration and innovation in production control.

4.2.3. Critical assessment of reinforcement learning challenges

This RL implementation faces typical multi-agent system challenges like computational complexity, nonstationarity, partial observability, and credit assignment (Ding and Dong 2020; Wong et al. 2022), impacting its effectiveness and generalizability.

Computational complexity stems from RL's low sample efficiency, needing extensive environment interactions to learn effective policies (Kakade 2003; Yu 2018). This is worsened in multi-agent systems where agents must learn concurrently, increasing computational load. While separate training for dispatching and routing agents helps, it doesn't completely resolve this. Larger systems might reduce complexity by subdividing into smaller subsystems to decrease the degree of dependencies of system elements, e.g. processes, resources and products.

Nonstationarity is significant as concurrent learning creates a constantly changing environment (Papoudakis et al. 2019), violating the Markov assumption and complicating RL algorithm convergence. We use an agent-environment cycle model to adapt agents' responses, but dynamic interactions still challenge stable, optimal policies.

Partial observability means agents act on local observations without global state access, leading to incomplete information (Kraemer and Banerjee 2016). We mitigate this with the PettingZoo library for multi-agent scenarios and Alpyne to interface Python with AnyLogic, enabling centralised observation sharing via the HiveContext framework. Agents can exchange route and station info, reducing partial observability uncertainty, though optimal performance remains challenging as agents infer the global state from local observations.

The credit assignment problem in multi-agent RL involves determining individual contributions to the joint reward (Minsky 1961). We use localised rewards for routing and dispatching agents for immediate feedback, reducing global reward attribution complexity. Rewards like *reward_target_distance* and *reward_reached_target* encourage efficient target reaching, while penalties like *reward_invalid* discourage invalid actions. *Reward_direction* manages travel direction, and *reward_pass* optimises AGV blockage behaviours.

Localized rewards can lead to selfish behaviours, undermining overall system performance. Reward shaping techniques, such as *reward_balance*, were used to promote collaborative behaviours, ensuring balanced processing of different part families and improving collective performance.

Explainable artificial intelligence is another critical issue. Techniques like human collaboration, visualisation, policy summarisation, query-based explanation, and verification are relevant (Wells and Bednarz 2021). Comparisons with rule-based methods, decision tree analysis of agent behaviour, and saliency maps highlighting influential input data parts have been used in RL for production control (Kuhnle et al. 2021).

In summary, while this RL application addresses some challenges through specific architectural and procedural choices, computational complexity, nonstationarity, partial observability, and credit assignment still impose constraints. Future work should enhance RL framework scalability and robustness with advanced techniques like hierarchical RL, improved communication protocols, sophisticated reward shaping for better agent collaboration, and generalising RL applicability to other production layouts.

5. Conclusion

The aim of this paper is to find a possible solution for the control of a flexible MSP by RL. For this purpose, the basics and state-of-the-art research on the control of comparable production systems by means of RL were presented first. It was found that controlling a MSP mainly involves two subtasks, dispatching and routing. Dispatching determines which next station to move to, and routing determines which specific movements will achieve that goal. Processing stations process the products at the time and in the order in which they arrive, and all of the complexity is represented in the transportation system.

This problem was formulated as a hierarchical multi-agent RL problem, in which each AGV is modelled by a dispatching agent as a manager agent and a routing agent as a worker agent. MSP was modelled as an agent-based event-driven simulation model in AnyLogic software. Care was taken to represent framework conditions such as AGV behaviour and size, signal exchange, and processing station behaviour as closely as possible to a real-world implementation. This ensured that the complexity of the control task described above was translated into the model.

In addition, it was taken into account that common interfaces and architectures were adhered to. In particular, it should be mentioned here that the simulation environment can be linked to various existing algorithms via the PettingZoo interface. The chosen solution for mapping multi-agent environments in AnyLogic models is modular and can be transferred to various other projects. The control of the agents in the simulation model was done using neural networks within the application of PPO and DQN algorithms.

For this purpose, different network structures were designed based on the current state of research. In order to determine the right network structure, but also the right algorithm and good hyperparameters, a large number of experiments were performed. In the process, various observations were made. For example, multi-agent RL is much more suitable for the control task than single-agent RL. The choice of algorithms, gamma value, and learning rate greatly impacted the training performance.

In addition, it was confirmed that the use of Fourier features added value to this coordinate-based problem. The use of Graph Attention Neural Networks turned out to be particularly effective for routing. Generally, in the context of multi-agent systems, it has proven effective to use network structures adapted to the problem's structure, such as Attention Neural Networks or networks of linear layers that selectively change dimension. It can be conjectured that all these findings apply in a similar form to similar control tasks. Better conclusions about transferability could be made if the selection of hyperparameters would basically follow a similar procedure.

Finally, the learned algorithm was compared with random dispatching and conflict-avoiding routing according to Möhring et al. (2004). First, it was found that the conventional approaches can poorly control a MSP without predefining the direction of travel under realistic conditions. The learned algorithm already solves this problem much better. Nevertheless, the selected MSP cannot be utilised to capacity with any of the algorithms shown. While good results are achieved for small fleet sizes, the transport performance per AGV collapses with increasing fleet size for all algorithms, and the number of blockages increases. Observing the system with the learned algorithms, it can be observed that partly complex situations are resolved intelligently, but partly simple decisions are made incorrectly. This shows that this form of control does have potential for optimisation.

For future work, various approaches for improvement can be identified. On the one hand, other network structures can be tested. For the Graph Attention Network that was used for routing, different variations of the first layers can be tried that transform the received data into a graph structure, making it easier for the network to access the information. In addition, the structure of the transportation network can be optimised to relieve traffic nodes or to specify individual directions of travel at important passages so that the risk of collision is reduced. Furthermore,

it would be interesting to find out whether communication between the agents leads to better results and which training algorithms can be used to optimise such a type of communication. In summary, the approach of controlling a MSP via hierarchical multi-agent RL has proven to be an improvement over conventional approaches, but not yet practical.

Disclosure statement

No potential conflict of interest was reported by the author(s).

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Notes on contributors



Lennart M. Steinbacher is a senior research associate at BIBA – Bremer Institut für Produktion und Logistik GmbH at the University of Bremen. He has earned a Master's degree in Industrial Engineering and Management from RWTH Aachen University, Germany. His research endeavours are focussed on addressing challenges in production and logistics systems by employing methodologies, including modelling, material flow simulation, operations research, data science, and machine learning.



Tobias Wegmann is currently an engineer specialising in structural planning at Mercedes-Benz AG in Bremen, Germany. He completed his Master's degree in Systems Engineering at the University of Bremen.



Michael Freitag is a full professor in the Faculty of Production Engineering of the University of Bremen, Germany, and Director of BIBA – Bremer Institut für Produktion und Logistik GmbH, Germany. He holds a Diploma degree in Electrical Engineering and a Doctoral degree in Production Engineering. His research interests include modeling, simulation, and optimization of complex production and logistics systems, the development of planning and control methods for logistic processes, and the automation of physical material flows through robots and flexible transport systems.

Data availability statement

The data that support the findings of this study are available from the corresponding author, Lennart M. Steinbacher, upon reasonable request. The software project used for this research is referenced (Wegmann 2023).

References

Altenmüller, Thomas, Tillmann Stüker, Bernd Waschneck, Andreas Kuhnle, and Gisela Lanza. 2020. "Reinforcement Learning for An Intelligent and Autonomous Production Control of Complex Job-Shops Under Time Constraints." *Production Engineering* 14 (3): 319–328. https://doi.org/10.1007/s11740-020-00967-8.

Bányai, Ágota, Béla Illés, Elke Glistau, Norge Isaias Coello Machado, Péter Tamás, Faiza Manzoor, and Tamás Bányai. 2019. "Smart Cyber-Physical Manufacturing: Extended and Real-Time Optimization of Logistics Resources in Matrix Production." *Applied Sciences* 9 (7): 1287. https://doi.org/10.3390/app9071287.

Bär, Schirin, Danielle Turner, Punit Mohanty, Vladimir Samsonov, Romuald Bakakeu, and Tobias Meisen. 2020. "Multi Agent Deep Q-Network Approach for Online Job Shop Scheduling in Flexible Manufacturing." In *2020 International Conference on Manufacturing System and Multiple Machines*, Tokyo, Japan, p. 1–9.

Bukchin, Joseph, Ezey M. Dar-El, and Jacob Rubinovitz. 2002. "Mixed Model Assembly Line Design in a Make-To-Order Environment." *Computers & Industrial Engineering* 41 (4): 405–421. https://doi.org/10.1016/S0360-8352(01)00065-1.

Damani, Mehul, Zhiyao Luo, Emerson Wenzel, and Guillaume Sartoretti. 2021. "PRIMAL: Pathfinding Via Reinforcement and Imitation Multi-Agent Learning – Lifelong." *IEEE Robotics and Automation Letters* 6 (2): 2666–2673. https://doi.org/10.1109/LSP.2016.

Ding, Zihan, and Hao Dong. 2020. "Challenges of Reinforcement Learning," edited by *Deep Reinforcement Learning*, edited by H. Dong, Z. Ding, and S. Zhang. Singapore: Springer. https://doi.org/10.1007/978-981-15-4095-0_7

Dittrich, Marc-André, and Silas Fohlmeister. 2020. "Cooperative Multi-Agent System for Production Control Using Reinforcement Learning." *CIRP Annals* 69 (1): 389–392. https://doi.org/10.1016/j.cirp.2020.04.005.

Echsler Minguillon, Fabio. 2020. *Prädiktiv-Reaktives Scheduling Zur Steigerung Der Robustheit in Der Matrix-Produktion*. Karsruhe: Doktorarbeit, Karlsruher Institut für Technologie.

Echsler Minguillon, Fabio, and Gisela Lanza. 2017. "Maschinelles Lernen in Der PPS: Konzept Für Den Einsatz Von Maschinellem Lernen Für Eine Matrix-Produktion." *Werkstattstechnik Online* 107 (9): 630–634. https://doi.org/10.37544/1436-4980-2017-09.

Feldkamp, Niclas, Sören Bergmann, and Steffen Strassburger. 2020. "Simulation-Based Deep Reinforcement Learning For Modular Production Systems." In *2020 Winter Simulation Conference (WSC)*, Orlando, FL, USA, 1596–1607. https://doi.org/10.1109/WSC48552.2020.9384089.

Filz, Marc-André, Johann Gerberding, Christoph Herrmann, and Sebastian Thiede. 2019. "Analyzing Different Material Supply Strategies in Matrix-Structured Manufacturing Systems." *Procedia CIRP* 81:1004–1009. https://doi.org/10.1016/j.procir.2019.03.242.

Greschke, Peter. 2016. *Matrix-Produktion Als Konzept Einer Taktunabhängigen Fließfertigung*. 1st ed. Essen, Germany: Schriftenreihe des IWF, Vulkan-Verlag.

Greschke, Peter, Malte Schönemann, Sebastian Thiede, and Christoph Herrmann. 2014. "Matrix Structures for High Volumes and Flexibility in Production Systems." *Procedia Cirp* 17:160–165. https://doi.org/10.1016/j.procir.2014.02.040.

Hayes, Robert H., and Steven C. Wheelwright. 1979. "Link Manufacturing Process and Product Life Cycles." *June* 2 (1): 1999.

Heger, Jens, and Thomas Voss. 2020. "Dynamically Changing Sequencing Rules with Reinforcement Learning in a Job Shop System With Stochastic Influences." In *2020 Winter Simulation Conference (WSC)*, Orlando, FL, USA, 1608–1618. https://doi.org/10.1109/WSC48552.2020.9383903.

Hofmann, Constantin, Nadine Brakemeier, Carmen Krahe, Nicole Stricker, and Gisela Lanza. 2018. "The Impact of Routing and Operation Flexibility on the Performance of Matrix Production Compared to a Production Line." In *Advances in Production Research: Proceedings of the 8th Congress of the German Academic Association for Production Technology (WGP)*, Aachen, November 19–20, 2018, 155–165. Springer International Publishing.

Hofmann, Constantin, Carmen Krahe, Nicole Stricker, and Gisela Lanza. 2020. "Autonomous Production Control for Matrix Production Based on Deep Q-Learning." *Procedia CIRP* 88:25–30. https://doi.org/10.1016/j.procir.2020.05.005.

Hu, Hao, Xiaoliang Jia, Qixuan He, Shifeng Fu, and Kuo Liu. 2020. "Deep Reinforcement Learning Based AGVs Real-Time Scheduling with Mixed Rule for Flexible Shop Floor in Industry 4.0." *Computers & Industrial Engineering* 149:106749. https://doi.org/10.1016/j.cie.2020.106749.

Kakade, Sham Machandranath. 2003. *On the Sample Complexity Of Reinforcement Learning*. University of London, University College London (United Kingdom).

Kamoshida, Ryota, and Yoriko Kazama. 2017. "Acquisition of Automated Guided Vehicle Route Planning Policy Using Deep Reinforcement Learning." In *2017 6th IEEE International Conference on Advanced Logistics and Transport (ICALT)*, Bali, Indonesia, 2017, 1–6. https://doi.org/10.1109/ICAdLT.2017.8547000.

Khairai, Kamarulzaman Mahmad, and Siti Nor Aisyah Khalil. 2024. "Line Balancing Study Using Value Stream Mapping Tool on Lean Manufacturing: A Case Study in An Electronic Industry." *Qomaruna* 1 (2): 55–64. https://api.semanticscholar.org/CorpusID:271436110.

Kim, Ju-Bong, Ho-Bin Choi, Gyu-Young Hwang, Kwihoon Kim, Yong-Geun Hong, and Youn-Hee Han. 2020. "Sortation Control Using Multi-Agent Deep Reinforcement Learning in N-Grid Sortation System." *Sensors (Basel, Switzerland)* 20 (12): 3401. https://pubmed.ncbi.nlm.nih.gov/32560217/.

Kool, Wouter, Herke Van Hoof, and Max Welling. 2018. "Attention, Learn to Solve Routing Problems!" arXiv preprint arXiv:1803.08475.

Koren, Yoram. 2010. *The Global Manufacturing Revolution: Product-Process-Business Integration and Reconfigurable Systems*. John Wiley & Sons.

Kraemer, Landon, and Bikramjit Banerjee. 2016. "Multi-Agent Reinforcement Learning as a Rehearsal for Decentralized Planning." *Neurocomputing* 190:82–94. https://doi.org/10.1016/j.neucom.2016.01.031.

Kuhnle, Andreas, Jan-Philipp Kaiser, Felix Theiß, Nicole Stricker, and Gisela Lanza. 2020. "Designing An Adaptive Production Control System Using Reinforcement Learning." *Journal of Intelligent Manufacturing* 32 (3): 855–876. https://doi.org/10.1007/s10845-020-01612-y.

Kuhnle, Andreas, Marvin Carl May, Louis Schäfer, and Gisela Lanza. 2021. "Explainable Reinforcement Learning in Production Control of Job Shop Manufacturing System." *International Journal of Production Research* 60 (19): 5812–5834. https://doi.org/10.1080/00207543.2021.1972179.

Luo, Shu. 2020. "Dynamic Scheduling for Flexible Job Shop with New Job Insertions by Deep Reinforcement Learning." *Applied Soft Computing* 91:106208. https://doi.org/10.1016/j.asoc.2020.106208.

May, Marvin Carl, Simon Schmidt, Andreas Kuhnle, Nicole Stricker, and Gisela Lanza. 2021. "Product Generation Module: Automated Production Planning for Optimized Workload and Increased Efficiency in Matrix Production Systems." *Procedia CIRP* 96:45–50. https://doi.org/10.1016/j.procir.2021.01.050.

Minsky, Marvin. 1961. "Steps toward Artificial Intelligence." *Proceedings of the IRE* 49 (1): 8–30. https://doi.org/10.1109/JRPROC.1961.287775.

Möhring, Rolf H., Ekkehard Köhler, Ewgenij Gawrilow, and Björn Stenzel. 2004. "Conflict-Free Real-Time AGV Routing." In *Operations Research Proceedings 2004: Selected Papers of the Annual International Conference of the German Operations Research Society (GOR)*, Jointly Organized with the Netherlands Society for Operations Research (NGB) Tilburg, September 1–3, 18–24. Springer Berlin Heidelberg.

Neumann, W. Patrick, and Per Medbo. 2017. "Simulating Operator Learning during Production Ramp-Up in Parallel Vs. Serial Flow Production." *International Journal of Production Research* 55 (3): 845–857. https://doi.org/10.1080/00207543.2016.1217362.

Nielsen, Christian P., Akshay Avhad, Casper Schou, and Elias Ribeiro da Silva. 2023. "Control System Architecture for Matrix-Structured Manufacturing Systems." *Computers in Industry* 146:103851. https://doi.org/10.1016/j.compind.2023.103851.

Panzer, Marcel, and Benedict Bender. 2022. "Deep Reinforcement Learning in Production Systems: A Systematic Literature Review." *International Journal of Production Research* 60 (13): 4316–4341. https://doi.org/10.1080/00207543.2021.1973138.

Papoudakis, Georgios, Filippos Christianos, Arrasy Rahman, and Stefano V. Albrecht. 2019. "Dealing with Non-Stationarity in Multi-Agent Deep Reinforcement Learning." arXiv preprint arXiv:1906.04737.

Popper, Jens, Vassilios Yfantis, and Martin Ruskowski. 2021. "Simultaneous Production and AGV Scheduling Using Multi-Agent Deep Reinforcement Learning." *Procedia CIRP* 104:1523–1528. https://doi.org/10.1016/j.procir.2021.11.257.

Rai, Rahul, Manoj Kumar Tiwari, Dmitry Ivanov, and Alexandre Dolgui. 2021. "Machine Learning in Manufacturing and Industry 4.0 Applications".

Renna, Paolo. 2023. "Performance Analysis of Matrix-Structured Manufacturing Systems Compared to Dedicated Flow and Reconfigurable Manufacturing Lines." *Production*

Engineering 18 (3–4): 625–642. https://doi.org/10.1007/s11740-023-01243-1.

Sartoretti, Guillaume, Justin Kerr, Yunfei Shi, Glenn Wagner, T. K. Satish Kumar, Sven Koenig, and Howie Choset. 2019. "Primal: Pathfinding Via Reinforcement and Imitation Multi-Agent Learning." *IEEE Robotics and Automation Letters* 4 (3): 2378–2385. https://doi.org/10.1109/LSP.2016.

Schönemann, Malte, Christoph Herrmann, Peter Greschke, and Sebastian Thiede. 2015. "Simulation of Matrix-Structured Manufacturing Systems." *Journal of Manufacturing Systems* 37:104–112. https://doi.org/10.1016/j.jmsy.2015.09.002.

Tancik, Matthew, Pratul P. Srinivasan, Ben Mildenhall, Sara Fridovich-Keil, Nithin Raghavan, Utkarsh Singhal, Ravi Ramamoorthi, Jonathan T. Barron, and Ren Ng. 2020. "Fourier Features Let Networks Learn High Frequency Functions in Low Dimensional Domains." Accessed December 17, 2022. https://arxiv.org/pdf/2006.10739.

Tompkins, James A. 2010. *Facilities Planning*. 4th ed. Hoboken, NJ: John Wiley & Sons. http://www.loc.gov/catdir/enhancements/fy1011/2010455781-d.html.

Vaswani, Ashish, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. "Attention is All You Need." *Advances in Neural Information Processing Systems* 30, 31st Conference on Neural Information Processing Systems. CA, USA: Long Beach.

Waschneck, Bernd, Andre Reichstaller, Lenz Belzner, Thomas Altenmuller, Thomas Bauernhansl, Alexander Knapp, and Andreas Kyek. 2018. "Deep Reinforcement Learning for Semiconductor Production Scheduling." In *2018 29th Annual SEMI Advanced Semiconductor Manufacturing Conference (ASMC)*, Saratoga Springs, NY, USA, 301–306. https://doi.org/10.1109/ASMC.2018.8373191.

Wegmann, Tobias. 2023. "Production Control with Reinforcement Learning for a Matrix-Structured Production System." https://github.com/tobihoernchen/thesis.

Wells, Lindsay, and Tomasz Bednarz. 2021. "Explainable AI and Reinforcement Learning – A Systematic Review of Current Approaches and Trends." *Frontiers in Artificial Intelligence* 4. https://doi.org/10.3389/frai.2021.550030.

Wong, Annie, Thomas Bäck, Anna V. Kononova, and Aske Plaat. 2022. "Deep Multiagent Reinforcement Learning: Challenges and Directions." *Artificial Intelligence Review* 56 (6): 5023–5056. https://doi.org/10.1007/s10462-022-10299-x.

Yu, Yang. 2018. "Towards Sample Efficient Reinforcement Learning." In *27th International Joint Conference on Artificial Intelligence, IJCAI 2018*, Stockholm, Sweden, 13 July 2018 through 19 July 2018; Code 140653.

Zhang, Xu-Jin, Hong-Yan Sang, Jun-Qing Li, Yu-Yan Han, and Peng Duan. 2022. "An Effective Multi-AGVs Dispatching Method Applied to Matrix Manufacturing Workshop." *Computers & Industrial Engineering* 163:107791. https://doi.org/10.1016/j.cie.2021.107791.

Zhang, Cong, Wen Song, Zhiguang Cao, Jie Zhang, Puay Siew Tan, and Xu Chi. 2020. "Learning to Dispatch for Job Shop Scheduling Via Deep Reinforcement Learning." *Advances in Neural Information Processing Systems* 33:1621–1632.

Zou, Wen-Qiang, Quan-Ke Pan, Tao Meng, Liang Gao, and Yu-Long Wang. 2020. "An Effective Discrete Artificial Bee Colony Algorithm for Multi-AGVs Dispatching Problem in a Matrix Manufacturing Workshop." *Expert Systems with Applications* 161:113675. https://doi.org/10.1016/j.eswa.2020.113675.

Zou, Wen-Qiang, Quan-Ke Pan, and M. Fatih Tasgetiren. 2021. "An Effective Iterated Greedy Algorithm for Solving a Multi-Compartment AGV Scheduling Problem in a Matrix Manufacturing Workshop." *Applied Soft Computing* 99:106945. https://doi.org/10.1016/j.asoc.2020.106945.

Appendices

Appendix 1. Agent environment components

Table A1. Components of agent environment cycles.

Variable	Description
S	Set of all possible states in the simulation, including positions, velocities, and statuses of AGVs and stations.
s_0	Initial state where AGVs start empty at random positions, and stations are empty, with production beginning at initial stations.
N	Number of agents, which varies depending on the configuration and can include both dispatching and routing agents.
A_i	Set of possible actions for each agent, such as moving in specific directions for routing or selecting stations for dispatching.
T_i	Transition function for each agent, defining how the agent's state changes in response to actions, such as updating positions and statuses.
P	Transition function of the environment, detailing how the environment state evolves based on agents' actions, including interactions between AGVs and stations.
R_i	Set of possible rewards and reward function for each agent, specifying the incentives for actions, such as minimising travel time or balancing station workloads.
Ω_i	Set of possible observations for each agent, encompassing information like positions and statuses of nearby AGVs and stations.
O_i	Observation function for each agent, determining how observations are gathered and processed to form the agent's view of the environment.
ν	Next agent function determining the sequence in which agents take their turns, managed by the simulation environment to ensure coordinated decision-making.

Appendix 2. Reward values

Table A2. Overview of rewards used during the experiments.

Reward	Task	Content	Value
reward_reached_target	Routing	The AGV reaches a destination node that is not its target	10
reward_wrong_target	Routing	The AGV reaches a destination node that is not its target	−1
reward_invalid	Routing	The AGV chooses a direction that is not possible	−0.1
reward_target_distance	Routing	Scaling factor for distance reward. If < 0, generates a positive value if the AGV gets closer to the target and a negative value if it moves away from it.	−1, 0
reward_removed_for_block	Routing	The AGV is involved in a block	−5, −10, −20∗
reward_direction	Routing	The AGV moves against the intended direction of travel	−0.1∗, 0
reward_pass	Routing	The AGV waits and lets another AGV pass	0, 0.2∗
reward_duration	Routing and Dispatching	The AGV has to wait in the current segment (livelock assumed)	−0.5∗, 0
reward_accepted_in_station	Dispatching	The AGV gets loaded or unloaded at the target station	0, 2∗
reward_declined_in_station	Dispatching	The AGV does neither get loaded or unloaded at the target station	−1
reward_geo_operation	Dispatching	At the target station, the part gets a geo process	0∗, 1
reward_respot_operation	Dispatching	At the target station, the part gets a respot process	0∗, 1
reward_rework_operation	Dispatching	At the target station, the part gets a rework process	0∗, 1
reward_part_completed	Dispatching	The part gets completed at the target station	0∗, 5
reward_reduce	Dispatching	Small negative reward for each routing step taken	−0.02
reward_balance	Dispatching	Production of front and rear end gets out of balance	−5, 0∗

Note: Values marked with an Asterisk, have been used for the final experiments.